

Activation Sparsification in Transformers: Pressure, Thresholding, and Site Placement

Working title — introduction and related work — 5 September 2026

1 Introduction

Activation sparsity can reduce language-model inference cost when specialized kernels skip products and weight accesses associated with zero activations. Recent work shows that useful sparsity can be exposed in pretrained models through thresholding or encouraged during training, with measured acceleration in both cases (Liu et al., 2024; Cetin et al., 2026). These successes create a practical design problem: how should a researcher combine and place sparsification interventions to obtain more useful sparsity at an acceptable quality cost? Answering this question matters when choosing which components of a recipe to retain, where to invest additional training, and which operations warrant sparse-kernel development.

The available approaches act on different parts of this problem. Post-hoc thresholding exploits the tolerance of an existing representation; activation regularization changes what the model learns; and broader site placement exposes additional operations to sparsification. TEAL, ProSparse, and Q-Sparse establish the effectiveness of these approaches and investigate choices within them (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). These findings motivate examining the additional value of pressure as thresholding and placement change. An intervention that helps on its own may add little to another intervention, or impose a different quality cost when applied elsewhere. Without knowing these dependencies, an improved sparsity fraction gives limited guidance about why a recipe works or how to adapt it to a different model.

We study this problem through three separately controllable design choices: *pressure*, which encourages selected activations toward zero; *thresholding*, which sets values to exactly zero according to a gate; and *site placement*, which specifies where each acts. *The interventions are separately controllable, but their effects need not be separable.* Our central question is how their individual effects and interactions determine the quality–sparsity trade-off. In particular, does adding pressure become more or less useful as thresholding changes, and how does that response depend on where pressure and gates are applied?

Explaining that response requires connecting learned behavior to the computation it affects. A reduction in activation magnitude can leave the number of exact zeros unchanged; additional zeros can also be concentrated at sites that affect only a small share of the forward pass. Conversely, extending the gate set can expose additional operations to sparsification while changing the representation the model must learn. The empirical issue is how learning responds to these choices: when pressure produces additional zeros, how much quality that change costs, and which operations account for the result. Distinguishing these outcomes helps diagnose whether limited gains reflect the response to pressure, the computational reach of the chosen sites, or the cost of expanding the intervention.

To make that diagnosis model-wide, we pair observed zero-product counts with an architecture-dependent reach ceiling. R_{model} measures the fraction of counted forward matrix-multiplication products with a zero activation operand; $R_{\text{model}}^{\text{max}}$ describes the fraction reachable if the selected sites were entirely zero. We refer to the former as *model-wide sparsity*, distinguishing it from a local activation zero fraction. This accounting makes the location of sparsity consequential: the same local zero rate can affect different amounts of computation as the architecture or workload changes. It helps distinguish a change in learned sparsity from a change in the work reachable through the chosen sites. Direct timing then tests whether an implementation can exploit the observed zeros on hardware.

We organize matched pretraining comparisons around a *sparsification intervention ladder* (Figure 1), which adds gates and pressure progressively across feed-forward (FFN) and attention sites. Pythia-family models provide a common architecture for examining these choices and their behavior

across sizes (Biderman et al., 2023). We use the ladder to trace individual and joint intervention effects from learned activations to the operations they affect. A transferable explanation must account for changes in those effects at another model size, where both learned responses and computational reach can differ. An account that meets this requirement can guide how sparsification recipes are adapted, including when to strengthen pressure, change a gate, or extend placement.

2 Related Work

2.1 Sparsification interventions

Post-hoc thresholding reveals sparsity that an already learned representation can tolerate. CATS thresholds small-magnitude activations within gated FFNs, with both training-free and fine-tuning settings (Lee et al., 2024). TEAL extends magnitude thresholding to attention and FFN projections and uses block-wise greedy allocation to account for their different sensitivities (Liu et al., 2024). Its placement and allocation studies show why a uniform sparsity target need not be equally appropriate throughout a model. These methods provide a reference for sparsity available without retraining, while motivating the question of how training can change that tolerance.

Training-time interventions allow the representation to adapt to sparse computation. ReLU Strikes Back studies restoring sparse activations in language models (Mirzadeh et al., 2023); ProSparse combines ReLU, progressive ℓ_1 pressure, and threshold shifting during continued training (Song et al., 2024). ProSparse’s schedule ablations show that comparable FFN sparsity can carry different quality costs. Sparsing Law examines how sparsity depends on regularization, data, and model size (Luo et al., 2025), while Cetin et al. (2026) demonstrate highly sparse FFNs induced by ℓ_1 regularization during pretraining. Together, these studies establish pressure as a way to reshape the quality–sparsity trade-off. A local FFN zero fraction alone leaves a further accounting question: how much of the complete model’s computation do those zeros affect?

Extending sparsification beyond FFNs makes the choice of sites explicit. Q-Sparse applies top- k selection to attention and FFN projection inputs, reports overall sparsity and activated-parameter budgets, and ablates both the sparsifying nonlinearity and the straight-through estimator (Wang et al., 2024). Spark Transformer combines sparse FFNs with sparse attention through token-position selection and examines the role of its low-cost predictor (You et al., 2025). Projection-input sparsity and attention-position sparsity affect different work; our intervention ladder also considers feature coordinates in the internal attention operands. Building on the existing component studies, we examine how the additional effect of activation pressure depends on thresholding and placement. Relating those effects to model-wide product counts and topology-dependent reach connects the quality cost of an intervention to the computation its zeros can affect.

2.2 Inference speedup

Converting activation sparsity into speedup requires an execution strategy suited to the workload. TEAL accelerates batch-one autoregressive decoding, where skipping weight accesses is valuable, and reports up to $1.8\times$ speedup at 50% sparsity under its projection-based accounting (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication: their TwELL format packs sparse activations as a projection produces them and supports fused FFN computation, reducing the overhead of materializing and consuming the sparse representation. They evaluate throughput for full-sequence forward passes, a different workload from token-by-token decoding. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). These results show why the location and execution of sparsity matter together. Skipped products and memory traffic must repay indexing, packing, and launch overheads, while operations that remain dense limit the gain of the complete model. A sparsity percentage alone does not specify this balance.

This dependence makes kernel adaptation part of evaluating whether a sparsification method transfers. With larger batches, different token masks can reduce the weight-loading savings available from sparsity, as Spark’s batching analysis illustrates (You et al., 2025). Block Q-Sparse introduces structure into the mask to accommodate batched execution (Wang et al., 2024); TwELL instead makes packing and fusion compatible with tiled computation (Cetin et al., 2026). Tensor shape, sparsity structure, and data layout therefore matter alongside the zero fraction. Iterative kernel generation with execution and profiling feedback offers an emerging tool for exploring such adaptations (Ouyang et al., 2025; Luo et al., 2026). Establishing a benefit still requires numerical correctness and complete-model timings that include selection and conversion costs on the target workload. These measurements test whether intervention-induced zero-product opportunities translate into reduced execution time after sparse-execution overheads are included.

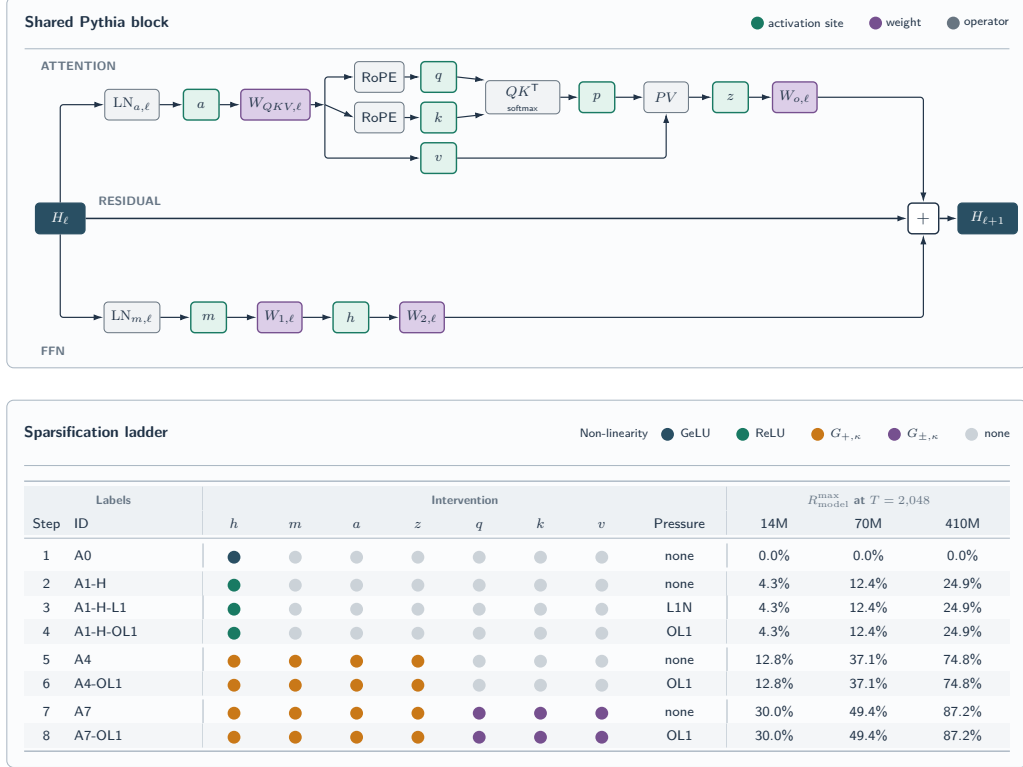


Figure 1: Sparsification intervention ladder and the shared Pythia block. The map distinguishes gate placement, gate family, and pressure method. The ceiling columns are analytic all-zero reach fractions at $T = 2,048$; they are neither observed sparsity nor speedup. The diagram defines candidate conditions, not a claim that every condition was evaluated at every size. In the corrected A4-OL1 and A7-OL1 recipes, pressure covers the four and seven gated sites, respectively; comparing these recipes changes both sets. Query and key gates are after RoPE, and z is the context before the attention output projection.

References

- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2304.01373>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. *arXiv preprint arXiv:2404.08763*, 2024. URL <https://arxiv.org/abs/2404.08763>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liqun Deng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing Law: Towards Large Language Models with Greater Activation Sparsity. *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/luo25i.html>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv preprint arXiv:2502.10517*, 2025. URL <https://arxiv.org/abs/2502.10517>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.